

COS 214 Report

<https://docs.google.com/document/d/1idU-lCoytu958DNpKCA-o5FY7hsPzeQXZI9UZDgsEA/edit?usp=sharing>

<https://github.com/WispyShrek/Project>

Lisa Kotzee 24595307

Marco de Wit 22811975

Cherise Heyl 21434019

Jonty Honey 23536862

Flip Venter 23692619

Research Brief:

The management of a nursery involves multiple components, such as the care of the plants housed in the nursery, inventory management and staff responsibilities. For effective and efficient management of a nursery, plants must remain healthy and customers must be assisted in a timely and respectful manner.

Key Components Of a Nursery:

1. **Plants and growing areas:** Organised into different garden types (Sunny, Partial sun and Shady) and greenhouses (which allow for a more controlled environment such that more specialised plants can be planted)
2. **Staff:** Divided into two primary roles – customer assistants and plant caretakers. Customer assistant staff interact with customers, handling the sale of plants and general queries regarding plants, while plant caretakers look after the plants and gardens in which plants are grown.
3. **Inventory:** Keeps record of all available stock, such as plants that have matured and can be sold to customers.

Plant Types and Care:

Currently the plants in the system do not align with their real world counterparts, but the general principles of plant care still apply.

- **Plants that reside in sunny gardens:** These plants should be placed in gardens that receive about six hours of sunlight a day, if not (as per the implementation) the plants will die.
- **Plants that reside in shady gardens:** These plants should receive no more than 2-3 hours of sunlight a day.
- **Plants that reside in partially sunny gardens:** These plants should be placed in gardens that receive less than 6, but more than 4 hours of sunlight a day.

Design influences:

The research done into plant care and nursery operations inspired role separation between staff members and environmental categorisation (the different types of gardens and the greenhouses). This allows for a clear and structured approach to the system, mimicking the real life operations of a nursery.

Assumptions:

- ❖ Plants can remain in the inventory indefinitely until they are sold.
- ❖ The inventory can be infinitely large, with no restrictions on the amount of plants that can be stored within
- ❖ The nursery can hold infinitely many gardens, staff members, plants and greenhouses

Functional Requirements:

- FR 1: The system must be able to allow the creation of a nursery by a user.
- FR 2: The user must be able to use the system to buy seedlings from a shop
- FR 3: The system must provide different types of seedlings with different attributes and needs
- FR 4: The user must be able to use the system to plant seedlings in unoccupied plots of land
- FR 5: The system must provide an interface for the user to take care of seedlings (watering, managing lighting etc.)
- FR 6: The system must cause seedlings to grow and mature into plants
- FR 7: The system must provide a mechanism for plants to react to environmental changes according to their attributes
- FR 8: The system must allow the user to modify base plants to add features and attributes
- FR 9: The system must make upgrades that affect the environment available to the user.
- FR 10: The system must allow the user to use sensors to monitor the environment
- FR 11: The system must provide an interface for the user to sell plants to customers
- FR 12: The system must simulate different times of day, weather conditions etc.

Non-functional requirements:

- NR 01: Performance: the system shall respond to button click events by the user in under 0.3 seconds. Load times for different interfaces within the system should be under 5 seconds to ensure that the user remains engaged.
- NR 02 :Usability: The system shall provide intuitive user interfaces for the management of the nursery, including tooltips as well as ways to quickly navigate to any part of the system.
- NR 03: Resiliency: The system shall be resistantDesifailurenouser input should not cause crashes, and sudden shutdown should not cause file corruption. High input volume will cause input to be buffered and handeled.

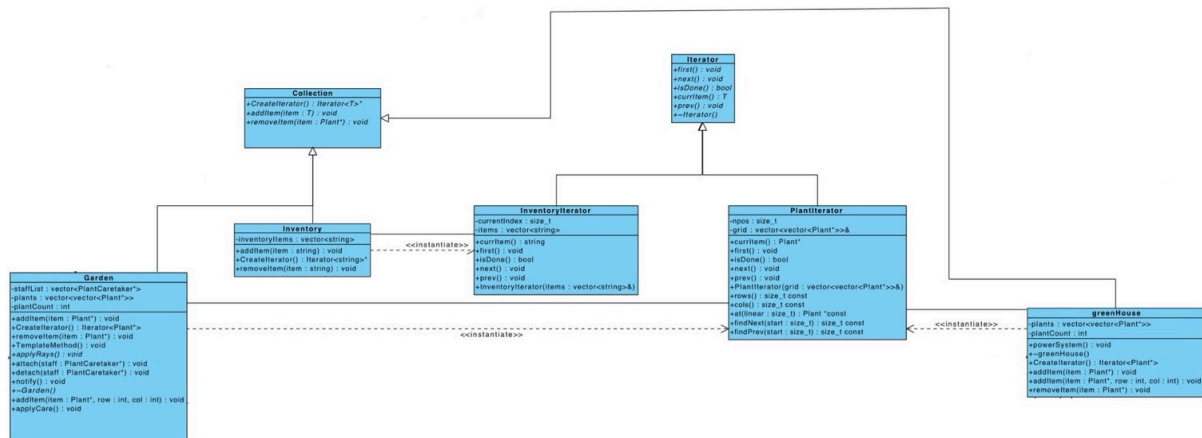
Design Patterns Used:

- **Factory Method**

- Prototype
- Iterator
- Mediator
- Observer
- Command
- Singleton
- Template
- Strategy
- Memento
- State
- Decorator

Design Pattern Application:

Iterator (Behavioural):



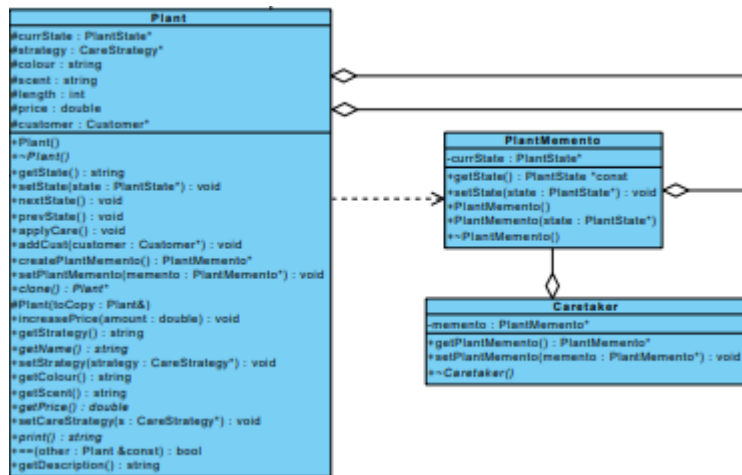
The Iterator Design Pattern was chosen to iterate over plants in gardens and greenhouses as well as items that have been stored in the inventory.

The Iterator Design Pattern suited this application perfectly, as general methods (such as moving to the next or previous item, removing an item and accessing an item) could be defined in an abstract class and then implemented specifically according to the needs of the specific collection.

The inventory stores a vector of strings, so the inventory iterator must account for strings and vectors, while gardens and greenhouses store plants, so the plant iterator must account for iteration over plants that are stored in a 2D matrix.

The Iterator Design Pattern addresses the fourth functional requirement: *"The user must be able to use the system to plant seedlings in unoccupied plots of land."*, and assists in the implementation of other design patterns.

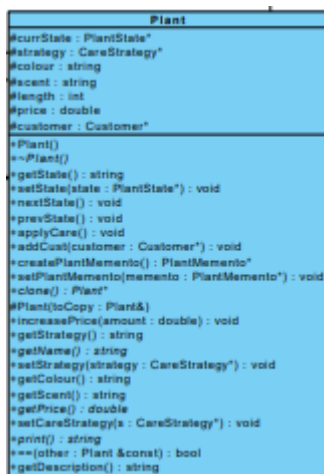
Memento(Behavioural):

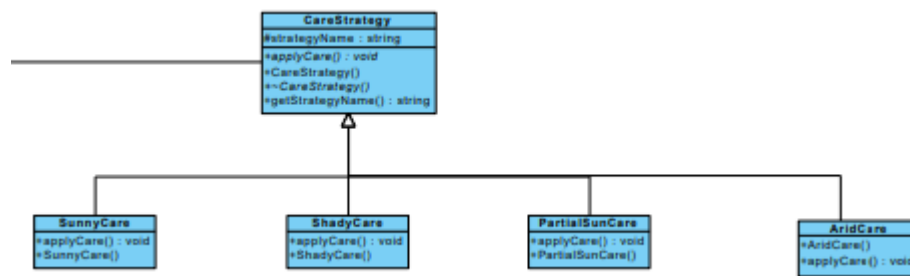


The Memento pattern is a behavioral pattern that allows an object's state to be saved and restored without revealing the details of its implementation. In our project, this pattern is crucial for fulfilling a key non-functional requirement and also provides the foundation for potential user-facing features.

The primary justification for using the Memento pattern is to satisfy NR 03: Resiliency. This requirement states, *"The system shall be able to recover from failures smoothly by making small save files, such that user data is not lost in the event of a crash or unexpected user program exit."* The Memento pattern provides the exact method needed to implement this save/load functionality. In our implementation the Originator participant is represented by the Plant class, the Memento participant is represented by the PlantMemento class and the Caretaker participant is represented by the Caretaker class.

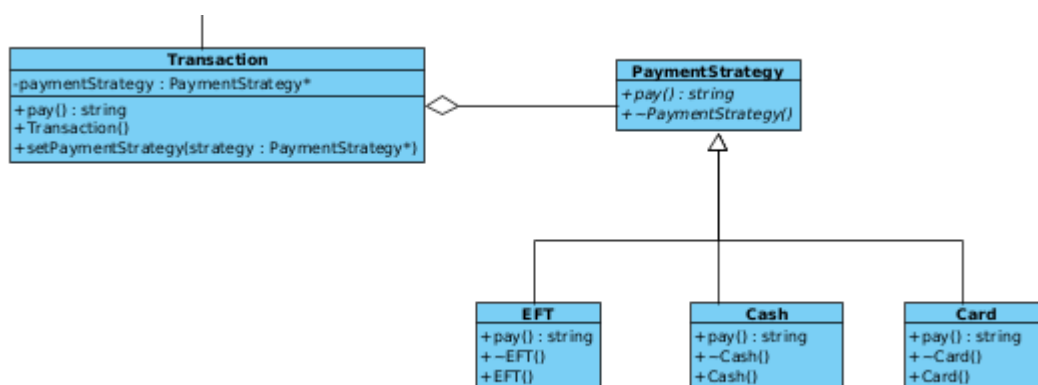
Strategy(Behavioural):





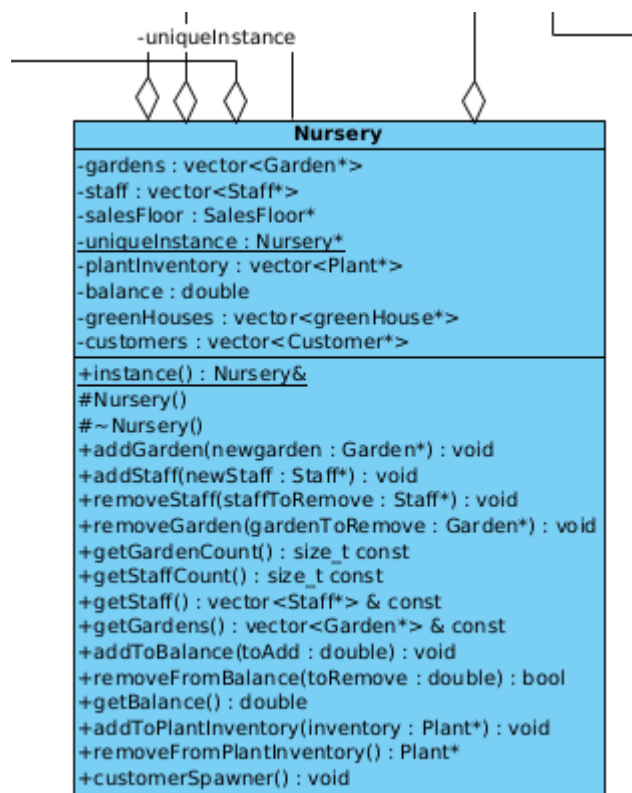
In our project, plants have distinct environmental needs, preferring either full sun, full shade, or partial sun. This required us to implement different ways to care for the plants based on these preferences. A simple approach would have been to write a large if-else statement inside the Plant class's `applyCare()` method. However, this would violate the Open/Closed Principle, as adding a new care method would require modifying the existing Plant class, making the design inflexible.

To avoid this, we made use of the Strategy design pattern, which allows us to encapsulate different care strategies in separate classes and make them interchangeable. Our implementation consists of three main components: the Context participant, represented by the Plant class; the Strategy participant, represented by the abstract **CareStrategy** class; and the Concrete Strategy participants, represented by our **SunnyCare**, **ShadyCare**, and **PartialSunCare** classes. The concrete strategies inherit from the **CareStrategy** interface and each implement their own version of the `applyCare()` method. This implementation is highly flexible, since new strategies can be introduced by simply adding a new Concrete Strategy class, thereby adhering to the Open/Closed Principle and this design directly fulfills functional requirement FR3, which requires that different plants have different needs, by allowing each Plant object to be assigned a specific **CareStrategy**. It also supports FR5, which requires an interface for the user to take care of seedlings, as the `applyCare()` method acts as this interface, delegating the actual work to the selected strategy.



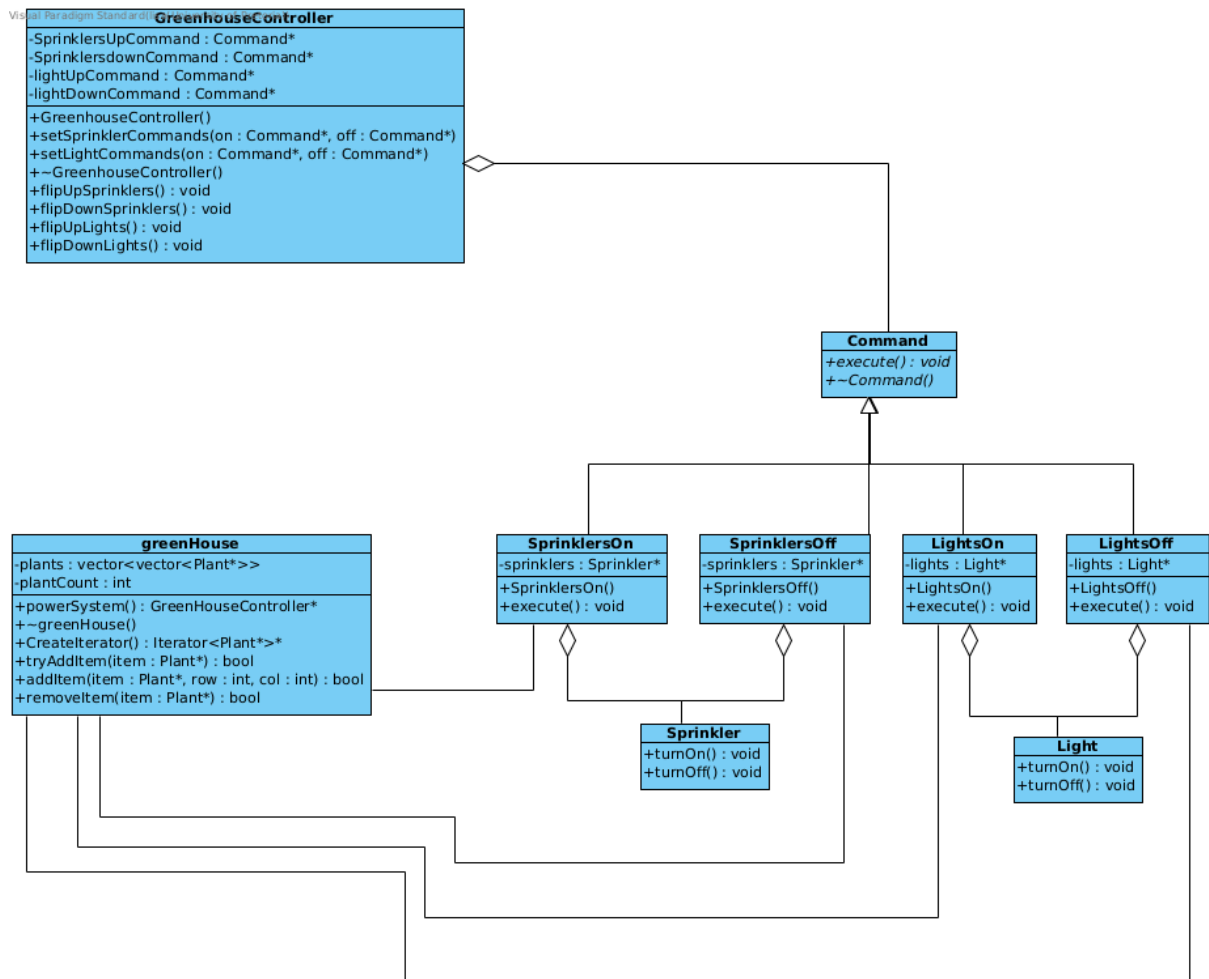
In addition to plant care, we also applied the Strategy design pattern to manage different payment methods. This implementation provided the same benefits of flexibility and adherence to the Open/Closed Principle as seen with the care strategies. For our payment system, the **Transaction** class acts as the Context participant, holding a reference to a payment algorithm. The abstract **PaymentStrategy** class serves as the Strategy interface, defining a common `pay()` method. Finally, the **EFT**, **Card**, and **Cash** classes are the Concrete Strategies, each providing a specific implementation of the `pay()` method, which allows the payment behavior of a transaction to be changed dynamically.

Singleton(Creational):



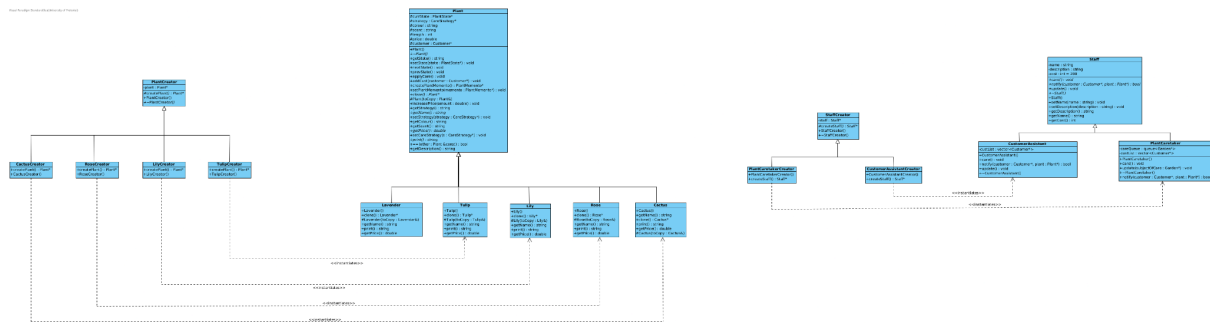
We used the Singleton design pattern for our Nursery class to make sure that we can only ever create one Nursery object in our whole program. This was important because the Nursery acts as the main hub for our application, holding all the lists of gardens and staff. If we could make more than one Nursery object, different parts of our code might be talking to different nurseries, which would lead to a lot of confusion and unexpected data. The Singleton pattern solves this by giving us one global way to get the nursery object by calling the static `Nursery::instance()` method, which is safer than just using a global variable. It guarantees that whenever any part of our code needs to work with the nursery, it always gets the exact same object, which keeps all our data consistent and in one place. The does not directly apply to the functional requirements but rather supports the functionality of other design patterns that adhere to them.

Command (Behavioural):



We use the Command design pattern to wrap each greenhouse action in a small class that does a clear thing, like TurnLightsOn, TurnSprinklersOn, TurnLightsOff, or TurnSprinklersOff. This means the UI never calls device methods directly; it just asks the command to execute (calls `execute()`), and the command handles the work. We used safe commands if one of the commands is not bound, meaning the controller won't crash, it just outputs a message of what needs to be set. Adding a new action is also easy, all you need is a new controller class and does not affect the controller that much, it stays small and easy to test. This leaves room for different features to be added on later if need be. Command address FR5 as it takes care of the seedlings and manages the different care actions that can be performed. FR12 is also addressed since the executed commands affect the climate inside of a greenhouse.

Factory Method - Plants (Creational):



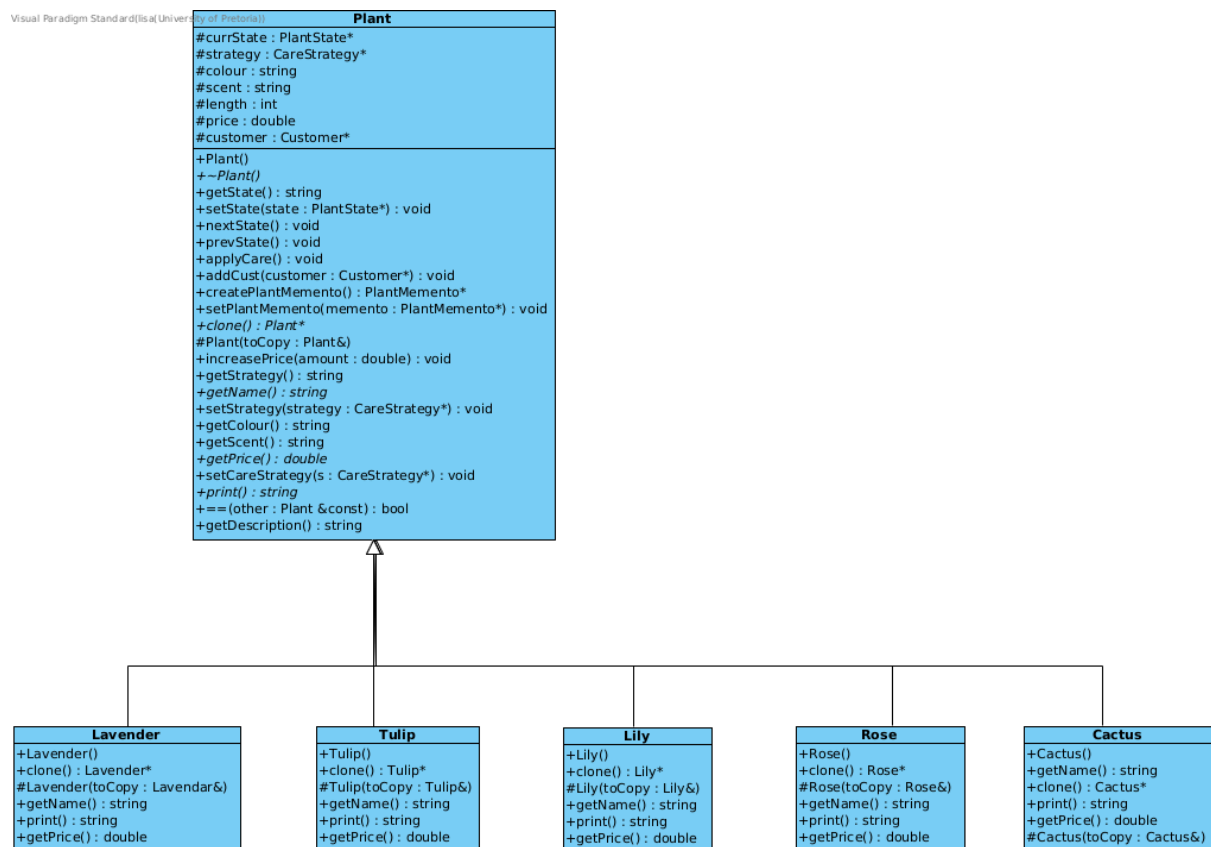
We use the Factory method design pattern so the code can ask for a plant by name without knowing the constructor/setup details. `PlantCreator` is the base and `LilyCreator`, `LavenderCreator`, `RoseCreator` and `TulipCreator` each build the correct subclass with certain properties (eg. price). This method also makes sure the correct `CareStrategy` is used so every plant starts valid and ready to use. Factory method addresses FR3 by ensuring that different seedlings come with different attributes that suit that type of plant.

Factory Method - Staff (Creational):

The Factory method design pattern builds staff objects with the right role and setup in one place, so the rest of the code does not need to worry about configuring them. `StaffCreator` is the base for `PlantCaretakerCreator` and `CustomerAssistantCreator`.

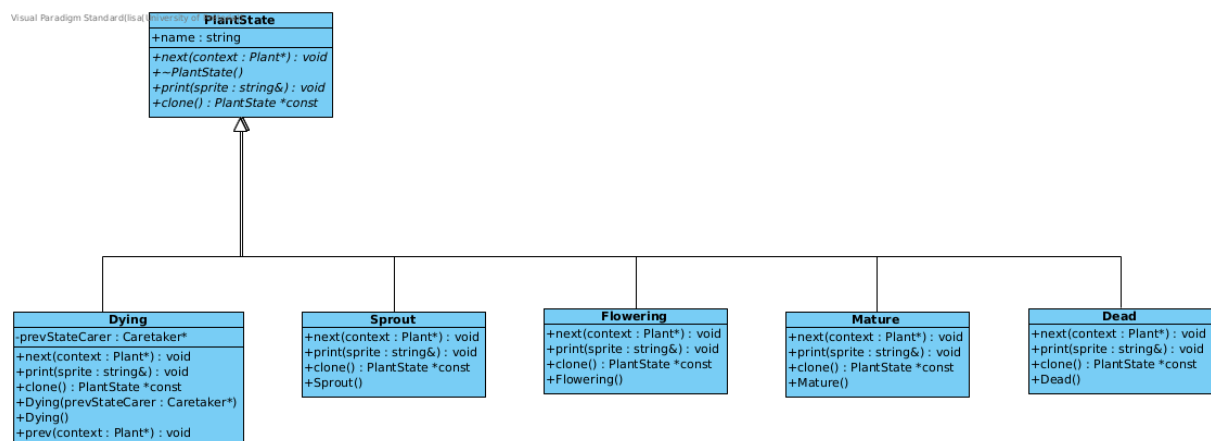
`CustomerAssistantCreator` returns a `CustomerAssistant` and `PlantCaretakerCreator` returns a `PlantCaretaker` that starts with care tools like `applyCare()`. This makes adding new roles easier later on as it keeps all the dependencies, rules and permissions together so nothing will break when a new role is added. Factory method addresses FR5 too as plant caretakers will apply care to plants in a dying state which helps bring them back to their original state, meaning the user can take care of the plant seedlings.

Prototype (Creational):



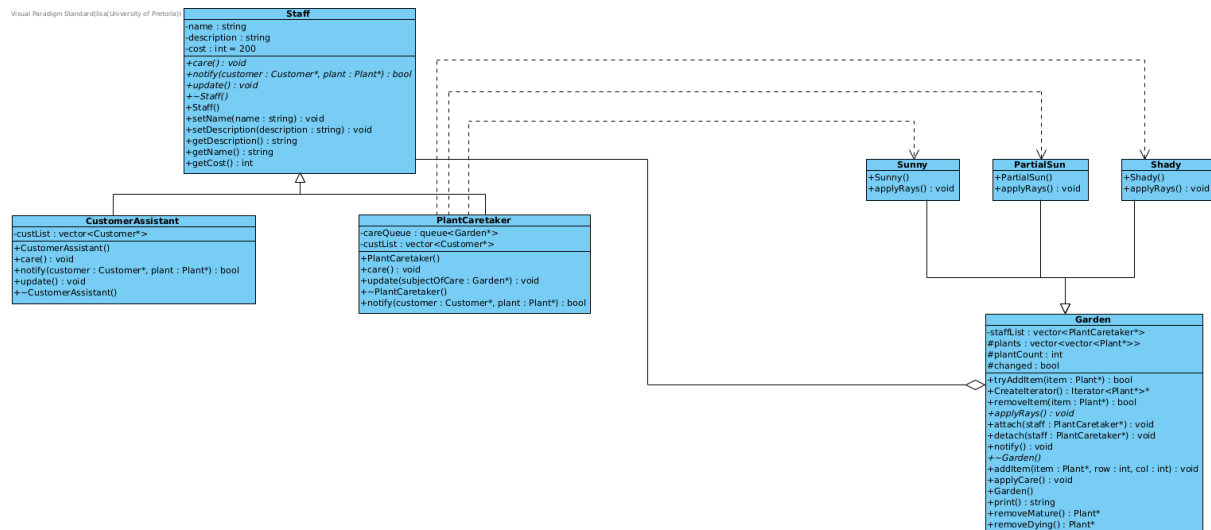
The Prototype design pattern lets us copy a ready-made objects quickly using clone(). For example, we can duplicate a configured staff member (and optionally plants) without redoing all the setup. This makes adding new plants of the same type so much easier. Prototype address FR2 and FR3 because we need many object to grow to sell later on so it allows us to sell multiple of the same plant.

State - (Behavioural):



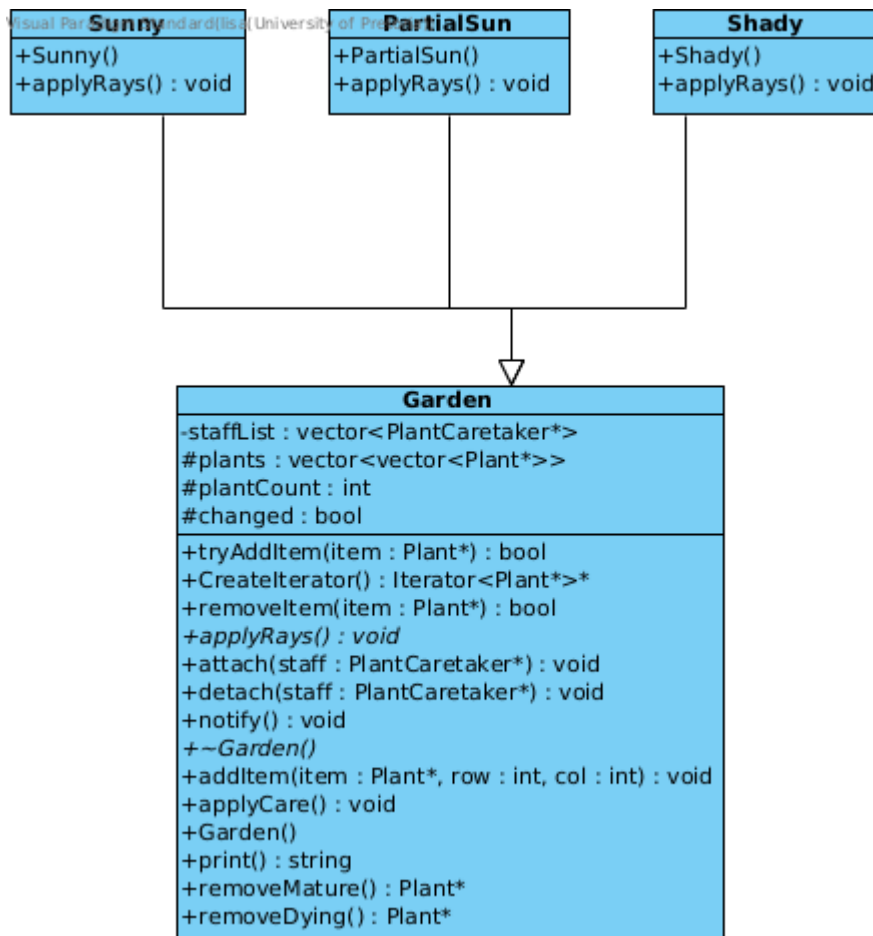
The State design pattern allows each plant to stage its own class: Sprout, Mature, Flowering, Dying, Dead. The current state decides what happens next. Healthy states can fall to Dying; if no care is given, Dying goes to Dead; if we do give care, we recover using Memento and this goes back to the state it was previously in before it fell into the dying state. Once a plant is mature it is ready to be sold. State addresses FR6 as it shows the different states a plant can grow into and also addresses FR7 as the current state of the plant affects how it reacts.

Observer-(Behavioral):



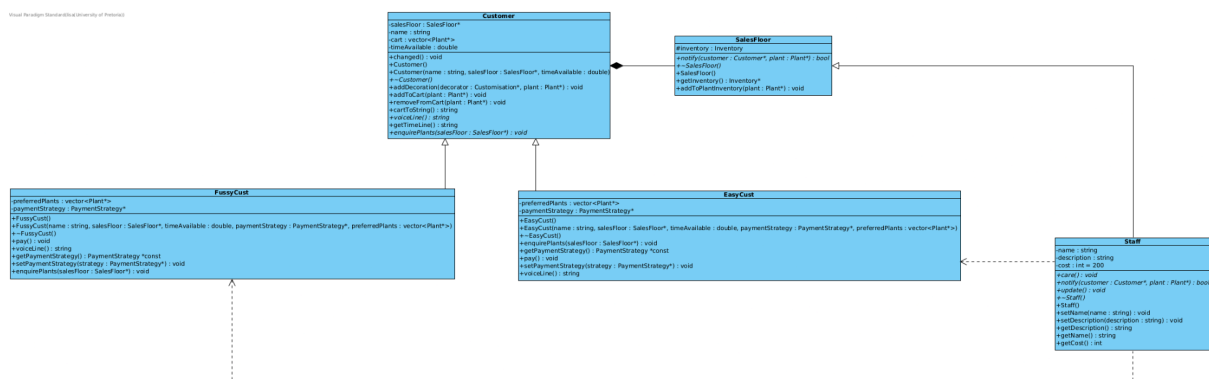
Observer was used to manage the interactions between plant caretakers and the gardens they are tending to. The gardens act as subjects and the plant caretakers are observers that can be attached or detached from the gardens. The user is given control over exactly which observer tends to any given garden as well as how many gardens each observer must tend to. The observer pattern addresses FR9 and FR10, as plant caretakers can be purchased as upgrades to the nursery, and also monitor the relevant gardens they are attached to on behalf of the user.

Template Method-(Behavioral):



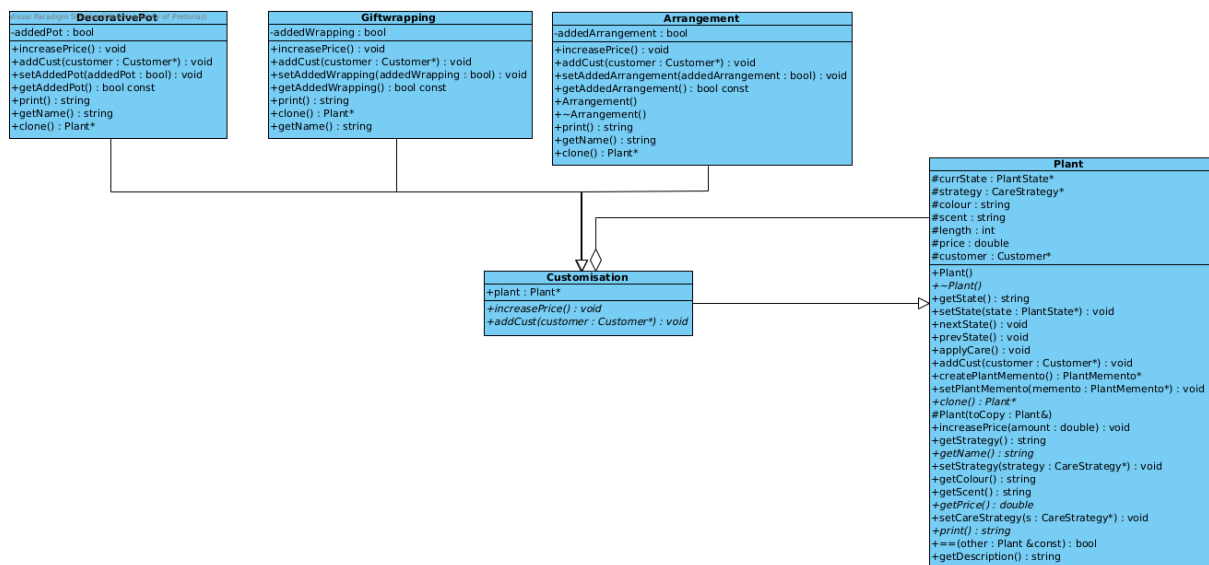
The Template Method design pattern allows for all gardens to share the outline of the tick() function. The garden (AbstractClass) defines the substeps, namely applyRays() as well as notify() and allows the ConcreteClasses to implement the specific functionality of the applyRays function whilst providing its own default implementation for notify. The Template method addresses FR6 and FR7 since the applyRays function is varied to cause plants that are placed within the wrong type of garden to wither away and die, and other plants to flourish when they are placed within the correct type of garden. The method makes use of state to apply changes to the plants in the relevant gardens.

Mediator-(Behavioral):



We used Mediator to manage communication between customers and staff within the nursery. The CustomerAssistant acts as the mediator that coordinates interactions, such as when a customer wants to buy or sell plants. Instead of having customers directly interact with multiple other classes, they send their requests to the mediator, which decides what needs to happen next. This keeps the system organised and prevents classes from being too dependent on each other. It also makes it easier to extend the system later with new customer or staff types. The Mediator pattern addresses FR11 as it provides the mechanism that enables plant sales to customers by coordinating between the relevant parts of the system.

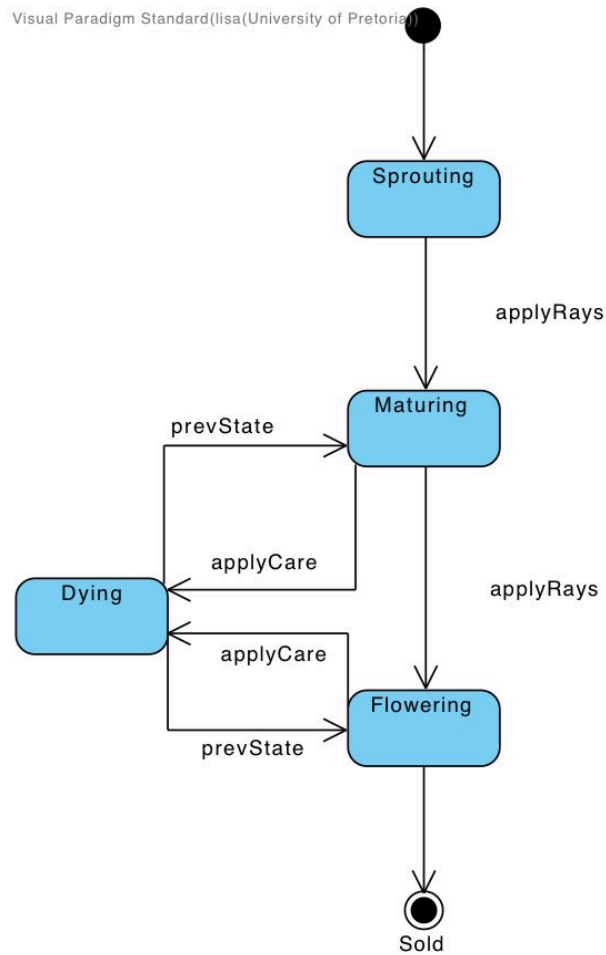
Decorator-(Structural):



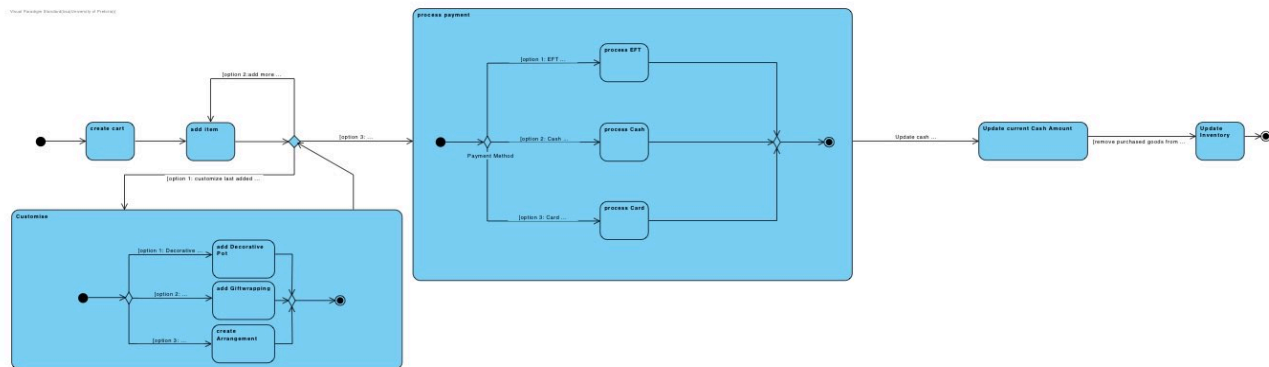
Decorator was used to allow for modification of plants by adding features that can also be mixed together. Each concrete decorator class adjusts the description and price while keeping the base class unchanged. The Decorator pattern addresses FR8, as it allows the user to modify and enhance plants with new attributes or features while maintaining consistent interactions with the base object.

UML Diagrams:

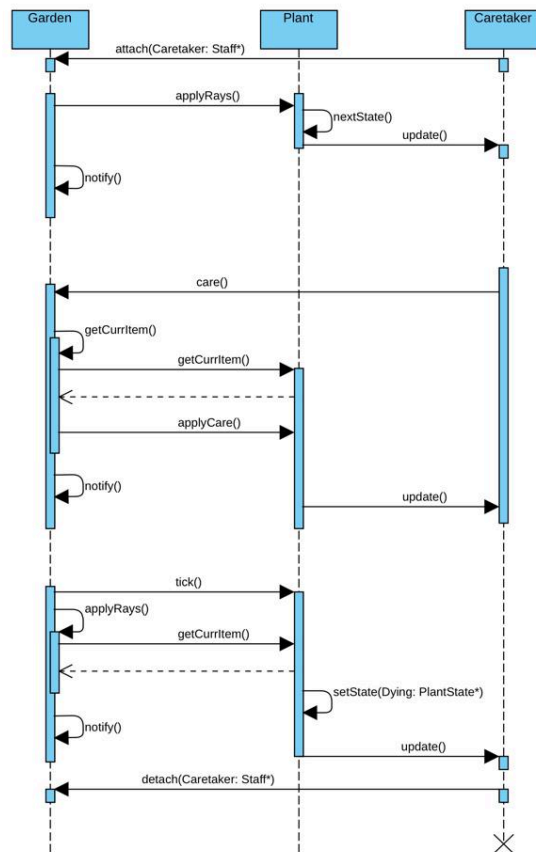
State Diagram:



Activity Diagram:



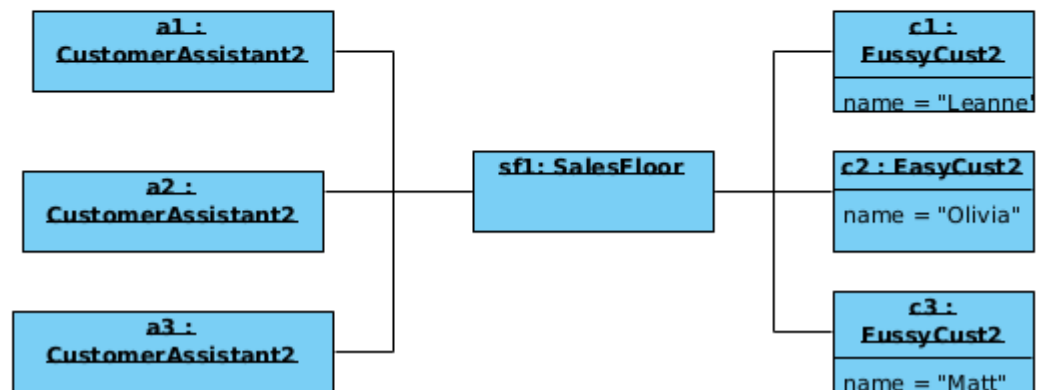
Sequence Diagram:



Object Diagram:

Visual Paradigm Standard (Marco de Wit (University of Pretoria))

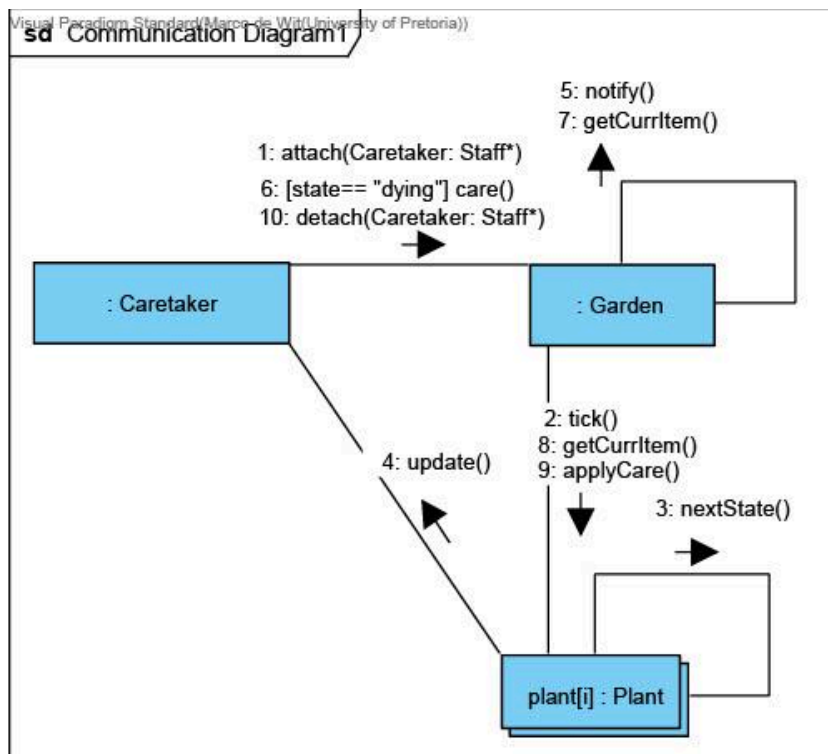
Mediator Snapshot: SalesFloor with Three Assistants and Three Customers



Communication Diagram:

Visual Paradigm Standard (Marco de Wit (University of Pretoria))

sd Communication Diagram 1



References:

Royal Horticultural Society (2019). *RHS - Inspiring everyone to grow / RHS Gardening*. [online] Rhs.org.uk. Available at: <https://www.rhs.org.uk/>.

IFAS Communications (2025). *Nursery and Greenhouse-Production Practices - Solutions for Your Life - University of Florida, Institute of Food and Agricultural Sciences - UF/IFAS*. [online] Ufl.edu. Available at: <https://sfyl.ifas.ufl.edu/agriculture/nursery-and-greenhouse-production-practices/> [Accessed 2 Nov. 2025].

Tilley, N. (2013). *What Is Full Sun And Tips For Full Sun Landscaping*. [online] Gardening Know How. Available at: <https://www.gardeningknowhow.com/garden-how-to/info/full-sun.htm> [Accessed 2 Nov. 2025].

Patterson, S. (2013). *What Is Partial Sunlight: Understanding Partial Sun Patterns*. [online] Gardening Know How. Available at: <https://www.gardeningknowhow.com/garden-how-to/info/what-is-partial-sunlight.htm>.

www.rhs.org.uk. (n.d.). *Shade gardening / RHS Gardening*. [online] Available at: <https://www.rhs.org.uk/garden-design/shade-gardening>.